Expressions and Arithmetic

Introduction to Computer Programming

Chandra Gummaluru

chandra-gummaluru.github.io

The Python **shell** runs one piece of code at a time and shows the result right away.

```
>>> 11 + 56
67
>>> 23 - 54
-31
>>> # This is a comment. Python ignores everything after the #
>>> 4 * 5
20
```

The >>> is Python's way of telling you that it's waiting for your **prompt**. You can think of the shell as a calculator of sorts.

calculator.py

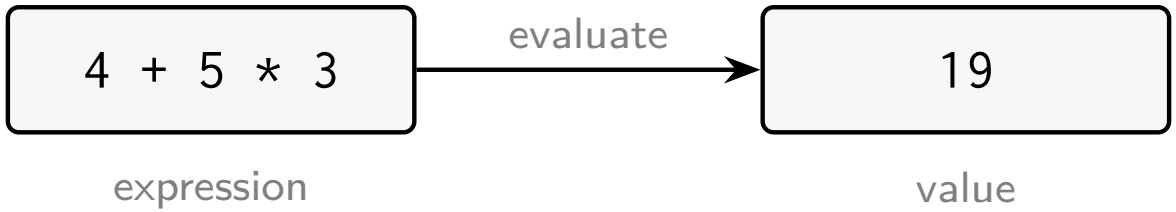
Program

```
1
```

Shell

```
>>> 11 + 56
67
>>> 8 / 4
2.0
>>> 8 // 5
1
>>> 8 % 5
3
>>> 2 ** 5
32
>>> type(3.0)
<class 'float'>
```

An **expression** is code that produces a single value. Python **evaluates** the expression to get that value.



expression	value
`2 + 4`	`6`
`8 / 5`	`1.6`
`(4 + 5) * 3`	`27`

Every value has a **type**. So far we have two types:

int	**float**
integer numbers	non-integer numbers
`3  -12  2026`	`3.0  -0.5  2.25`

We can check the type of a value using `type()`.

```
>>> type(3)
<class 'int'>
>>> type(3.0)
<class 'float'>
>>> 7 / 3
2.3333333333333335
```

arithmetic operators

An **operator** combines values to produce a new value.

operator	symbol	example	value
addition	`+`	`2 + 3`	`5`
subtraction	`-`	`6 - 2`	`4`
multiplication	`*`	`7 * 3`	`21`
exponentiation	`**`	`2 ** 5`	`32`
division	`/`	`5 / 2`	`2.5`
integer division	`//`	`5 // 2`	`2`
modulo (remainder)	`%`	`5 % 2`	`1`

Python has three ways to divide. Think of dividing 8 slices of pizza between 5 people.

division	**integer division**	**modulo**
the exact share	whole slices each	slices left over
`8 / 5` → `1.6`	`8 // 5` → `1`	`8 % 5` → `3`
`8 / 4` → `2.0`	`8 // 4` → `2`	`8 % 4` → `0`

What value does this expression produce?

```
20 / 4
```

A 5

✓ 5.0

C 4

D 0

operator precedence

When an expression has several operators, **precedence** decides which happens first, just like in math.

	operator	
highest	`( )`	parentheses
	`**`	exponentiation
	`-`	negation
	`*  /  //  %`	left to right
lowest	`+  -`	left to right

when in doubt, add parentheses; they also make your code easier to read

To evaluate an expression, find the operator with the highest precedence, evaluate it, and repeat.

```
5 + 2 ** 3 * 5
5 + 8 * 5
5 + 40
45
```

```
-10 * 3 + 5 ** 3
-10 * 3 + 125
-30 + 125
95
```

the operator evaluated at each step is in red

In which order are the operations evaluated?

```
1  7 / 3 + 4 ** 2
```

A. `**`, then `+`, then `/`

✓ `**`, then `/`, then `+`

C. `/`, then `+`, then `**`

D. `/`, then `**`, then `+`

What value does this expression produce?

```
2 + 3 * 4 ** 2
```

Answer: `50`

What value does this expression produce?

```
17 % 5 + 4 * 6 - 7
```

Answer: `19`

errors

Code can go wrong in two different ways.

syntax

the combination of symbols is not valid `Python`

semantics

the meaning of a valid combination of symbols is incorrect

When Python can't run your code, it tells you the **kind of error** and where it happened. Read the last line first.

```
>>> 3 +
SyntaxError: invalid syntax
>>> 4 / 0
Traceback (most recent call last):
  File "<pyshell#34>", line 1, in <module>
    4 / 0
ZeroDivisionError: division by zero
```

What happens when this is typed into the shell?

```
4 + 5) * 3
```

A It produces `27`

B It produces `9`

✓ `SyntaxError`

D `ZeroDivisionError`